

# وثيقة الهندسة المعمارية والتصميم البرمجي لنظام

## EduFinance

نظام إدارة الشؤون المالية التعليمية (Educational Finance Management System)

### 1. مقدمة النظام ونطاق العمل (System Introduction & Scope)

يمثل نظام **EduFinance** نموذجاً تطبيقياً متكاملًا لأنظمة سطح المكتب (Desktop Applications) الموجهة لأتمتة الحوكمة المالية داخل المؤسسات التعليمية. تم تطوير النظام بالاعتماد على لغة **Java** مع إطار العمل **JavaFX** لبناء الواجهات الرسومية، مستهدفًا تحقيق أعلى درجات الفصل بين منطق العمل وواجهة العرض. يكمن الهدف الجوهرى للنظام في استبدال الإدارة اليدوية التقليدية بألية رقمية متقدمة تدير محورين أساسيين: الشؤون المالية للطلاب (أقساط ورسوم دراسية) والشؤون المالية للمعلمين (مسيرات رواتب وأجور حصص).

#### 1.1 مجال عمل النظام والوظائف الأساسية

| الميزة الوظيفية                     | الوصف الهندسي والعملي                                                                                                                 |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| إدارة الطلاب (Student Management)   | القدرة الكاملة على إجراء عمليات العمليات الأساسية (إضافة، تعديل، حذف) لبيانات الطلاب وربطهم بالمراحل الدراسية المقابلة بشكل ديناميكي. |
| أقساط الطلاب (Student Installments) | تتبع العمليات المالية الخاصة بكل طالب، وحساب المتبقي تلقائياً بناءً على رسوم المرحلة الدراسية، مع توليد وصل دفع فوري بصيغة PDF.       |
| إدارة المعلمين (Teacher Management) | إدارة كاملة لملفات المعلمين وتحديد نوع التعاقد بدقة (نظام الأجر الثابت أو نظام الدفع مقابل الحصة).                                    |
| مسير الرواتب (Payroll System)       | معالجة وحساب مستحقات المعلمين استناداً إلى استراتيجية الدفع المحددة، وإصدار مسيرات رواتب رسمية ومؤرخة بصيغة PDF.                      |
| السجلات والتقارير المالية           | استعراض السجل المالي التاريخي والتعاملات السابقة لكافة الأطراف داخل النظام لضمان الشفافية والموثوقية.                                 |

| الميزة الوظيفية         | الوصف الهندسي والعملي                                                                                              |
|-------------------------|--------------------------------------------------------------------------------------------------------------------|
| إدارة الصلاحيات والوصول | نظام حماية ثنائي الأدوار (Admin / User) يقيّد الوصول للواجهات الحساسة والإعدادات المتقدمة بناءً على هوية المستخدم. |

## 1.2 البيئة التقنية والمكتبات المستخدمة

| التقنية / المكتبة | الإصدار  | الغرض المعماري من الاستخدام                                                                              |
|-------------------|----------|----------------------------------------------------------------------------------------------------------|
| Java SE           | 17 (LTS) | لغة البرمجة الأساسية للاستفادة من استقرار الميزات المتقدمة كالميزات غنية الكهولة ومحرك الآلة الافتراضية. |
| JavaFX            | 17.0.8   | بناء واجهة المستخدم الرسومية (GUI) وفصل التصميم المرئي عبر ملفات .FXML                                   |
| SQLite JDBC       | 3.45.1.0 | قاعدة بيانات محلية خفيفة الوزن مدمجة تضمن سرعة الاستعلام وسهولة النقل دون الحاجة لخادم مستقل.            |
| iText PDF         | 5.5.13.3 | توليد المستندات والتقارير المالية والوصلات بشكل فوري مع دعم كامل للخطوط والاتجاهات العربية (RTL).        |
| Maven             | x.3      | أداة إدارة بناء المشروع والتبعيات (Dependencies) بشكل قياسي وموحد.                                       |

## 2. النمط المعماري العام (MVC — Architectural Pattern)

- **طبقة النموذج (Model):** وتتمثل في مجلد كلاسات الكيانات البيانية (مثل Student, Teacher, Grade, Transaction, User). هذه الكلاسات مجردة من أي منطق يتعلق بالواجهات أو قواعد البيانات، ووظيفتها حفظ حالة البيانات وهيكلتها فقط.
- **طبقة العرض (View):** وتتمثل في ملفات واجهات التطبيق المكتوبة بصيغة FXML وملفات التنسيق المتتالي CSS. تحدد هذه الطبقة التخطيط المرئي وعناصر التحكم الرسومية دون احتواء أي كود برمجي منطقي.

- **طبقة التحكم (Controller):** كلاسات التحكم وتعمل كحلقة وصل تفاعلية، حيث تستقبل الأحداث (Events) من الواجهات الرسومية، وتقوم باستدعاء منطق العمل، والاستعلام من قاعدة البيانات عبر طبقة الوصول للبيانات، ومن ثم تحديث طبقة العرض ديناميكياً.

## 3. مبادئ البرمجة كائنية التوجه (OOP Principles)

### 3.1 التغليف (Encapsulation)

يتجلى مبدأ إخفاء البيانات وحمايتها من التعديل العشوائي الخارجي عبر جعل جميع المتغيرات والخصائص داخل كلاسات النماذج (Model Classes) تحمل محدد الوصول `private`. لا يمكن لأي كلاس خارجي أو متحكم الوصول لهذه البيانات إلا من خلال واجهة محكمة ومحددة تتمثل في دالات القراءة والتعديل (Getters & Setters).  
مثال برمجي من كلاس الطالب والمعلم:

```
// Student.java - حماية الخصائص ومنع التلاعب بها مباشرة
private int gradeId;
private double paidAmount;
private double remainingAmount;

public double getPaidAmount() {
    return paidAmount;
}

public void setPaidAmount(double paidAmount) {
    if(paidAmount >= 0) { // التغليف يسمح بوضع شروط تحقق صحة البيانات
        this.paidAmount = paidAmount;
    }
}
```

### 3.2 الوراثة (Inheritance)

تم بناء هيكلية مرنة تعتمد على الوراثة لتقليل تكرار الكود (DRY Principle) وتجميع الخصائص المشتركة في كلاسات عليا مجردة (Superclasses)، وتفريع كلاسات أبناء (Subclasses) ترث وتضيف ميزات الخاصة:

- الهرمية الأولى: الكلاس المجرد `Person` يمثل المفهوم العام للشخص (يحتوي على `id` و `name`). يرث منه كلاس `Student` (يضيف خصائص المرحلة والمبالغ المدفوعة) وكلاس `Teacher` (يضيف خصائص الراتب والقيمة والتعاقد).
- الهرمية الثانية: الكلاس المجرد `User` يمثل مستخدم النظام (يحتوي على الحساب وكلمة المرور والدور). يرث منه كلاس `Admin` (لتنشيط دور المدير) وكلاس `RegularUser` (لتنشيط دور المستخدم العادي).

```
// Admin.java - Super وراثه كلاس المستخدم وتمرير القيمة للـ
public class Admin extends User {
    public Admin(int id, String username, String password) {
```

```

        super(id, username, password, "ADMIN"); // استدعاء منشئ الأب
    }
}

```

### 3.3 تعدد الأشكال (Polymorphism)

يُعتبر تعدد الأشكال في وقت التشغيل (Runtime Polymorphism) عبر آليتي المراجع المشتركة وإعادة تعريف الدوال (Method Overriding) من أقوى التطبيقات البرمجية في المشروع ويظهر في موضعين هما:

أولاً: تعدد الأشكال في حساب المستحقات المالية للمعلم: يتعامل كلاس PayrollController مع كائن من نوع Teacher دون معرفة تفاصيل نوع عقده، وعند استدعاء الدالة calculatePay() يتغير السلوك ديناميكياً بحسب نوع العقد:

```

// داخل المتحكم يتم الاستدعاء بشكل موحد وثابت
finalPayAmount = selectedTeacher.calculatePay(sessions);

// يتغير السلوك بناءً على حالة الكائن الحالية Teacher داخل كلاس
@Override
public double calculatePay(int sessions) {
    if ("HOURLY".equalsIgnoreCase(payType)) {
        return sessions * payValue; // استراتيجية الحصة
    } else {
        return payValue; // استراتيجية الراتب الثابت
    }
}

```

ثانياً: تعدد الأشكال في جلسة المستخدم الحالية: يتم تخزين المستخدم الحالي للتطبيق في مرجع ثابت من النوع المجرد public static User currentUser؛، ولكن أثناء تشغيل التطبيق ونجاح الدخول، يحمل هذا المرجع كائناً حقيقياً إما Admin أو RegularUser، وتتعامل شاشات النظام مع المرجع المجرد لقراءة الصلاحيات بشكل مرن ومتعدد الأشكال.

### 3.4 التجريد والواجهات (Abstraction & Interfaces)

التجريد هو إخفاء التفاصيل المعقدة وتقديم واجهة عمل عامة. تم تطبيق هذا المفهوم من خلال الكلاسات التجريدية abstract class مثل Person و User لمنع إنشاء كائنات مباشرة منها لأنها تمثل مفاهيم عامة وليست كيانات عينية. كذلك، تم إدراج الواجهة الهندسية Payable لتكون عقداً برمجياً صارماً يجبر أي فئة تقوم بتنفيذها على تقديم آلية لحساب المستحقات، مما يفصل منطق الحساب المالي عن كود المعالجة في الواجهات فصلاً تاماً.

```

public interface Payable {
    double calculatePay(int sessions); // عقد تصميمي مجرد
}

```

## 4. أنماط التصميم المطبقة (Design Patterns Analysis)

- **نمط (Singleton):** طُبِّق في كلاس DatabaseConnection لضمان فتح اتصال واحد مشترك وقابل لإعادة الاستخدام بقاعدة البيانات SQLite طوال دورة حياة النظام، مما يوفر موارد الذاكرة ويمنع تضارب العمليات.
- **نمط (Strategy Pattern):** تم تحقيقه عبر الواجهة Payable، حيث يمكن للنظام تبديل خوارزمية حساب المستحقات (ثابت أو بالحصة) في وقت التشغيل بسلاسة ودون الحاجة لتكرار جمل الاختصار الشرطية if/else المعقدة في طبقة التحكم.
- **نمط (Implicit Factory):** يتجلى في كلاس LoginController، حيث يتم إنشاء الكائن البرمجي المناسب (إما Admin أو RegularUser) ديناميكياً استناداً إلى قيمة دور المستخدم المسترجعة من قاعدة البيانات عند نجاح عملية تسجيل الدخول.
- **التحكم في الوصول المستند إلى الأدوار (RBAC):** نمط أمني يحدد صلاحيات كل مستخدم، حيث يقوم كلاس DashboardController بقراءة نوع الجلسة وإخفاء أو إظهار أزرار التحكم والواجهات الحساسة (مثل إعدادات المراحل والمستخدمين) بناءً على الدور الحالي.

## 5. هيكلية الطبقات وتدفق البيانات (Layered Architecture & Data Flow)

ينقسم النظام برمجياً إلى طبقات هرمية متسلسلة لضمان استقلالية كل جزء وسهولة صيانتها واختباره بشكل منفصل:

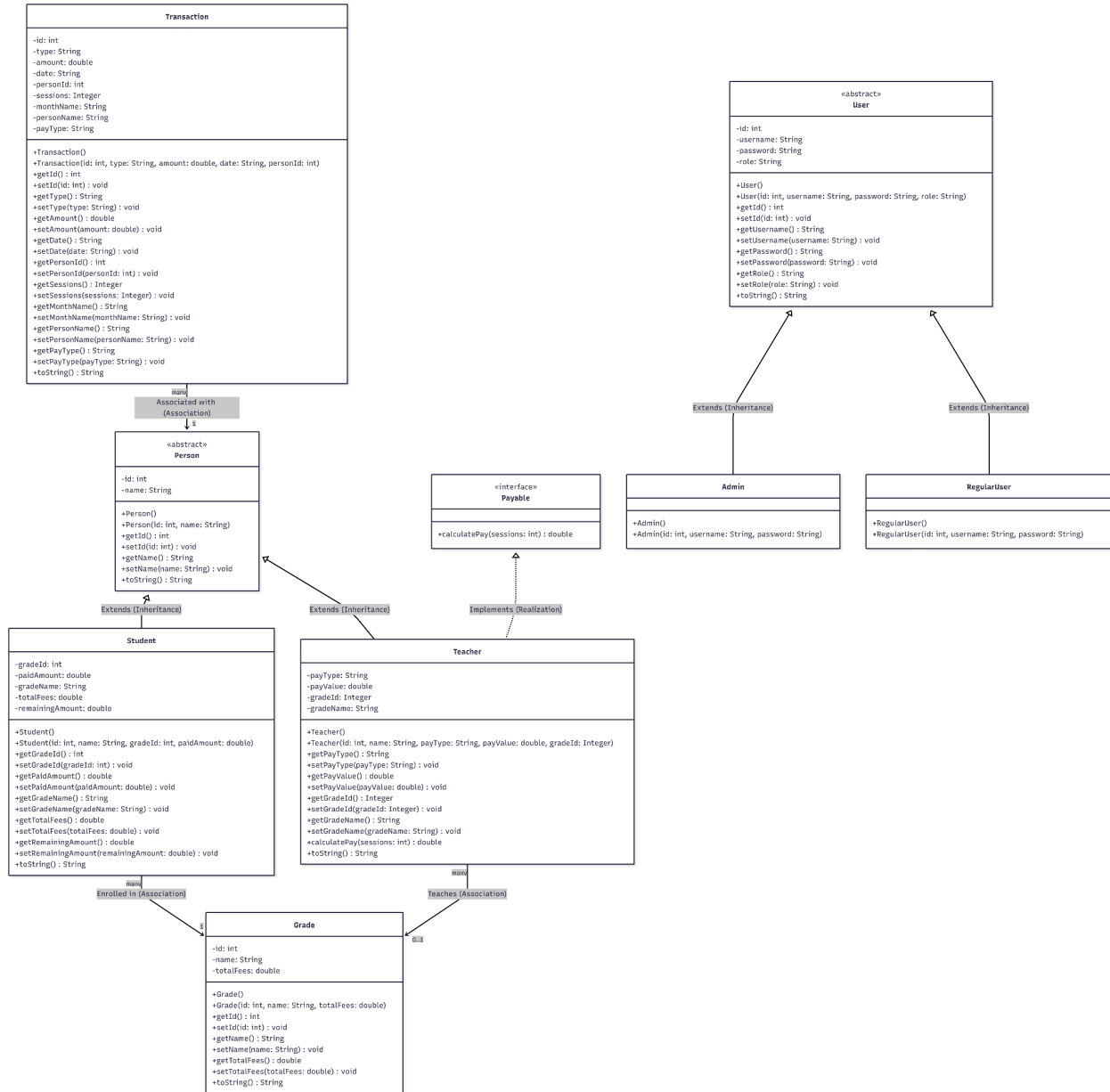
طبقة العرض (View) → طبقة التحكم والمنطق (Controller) → طبقة كائنات النماذج (Model) → طبقة الوصول لقاعدة البيانات (DAL) → طبقة الأدوات المساعدة (Utilities)

### 5.1 تدفق المعاملات المالية الحساسة

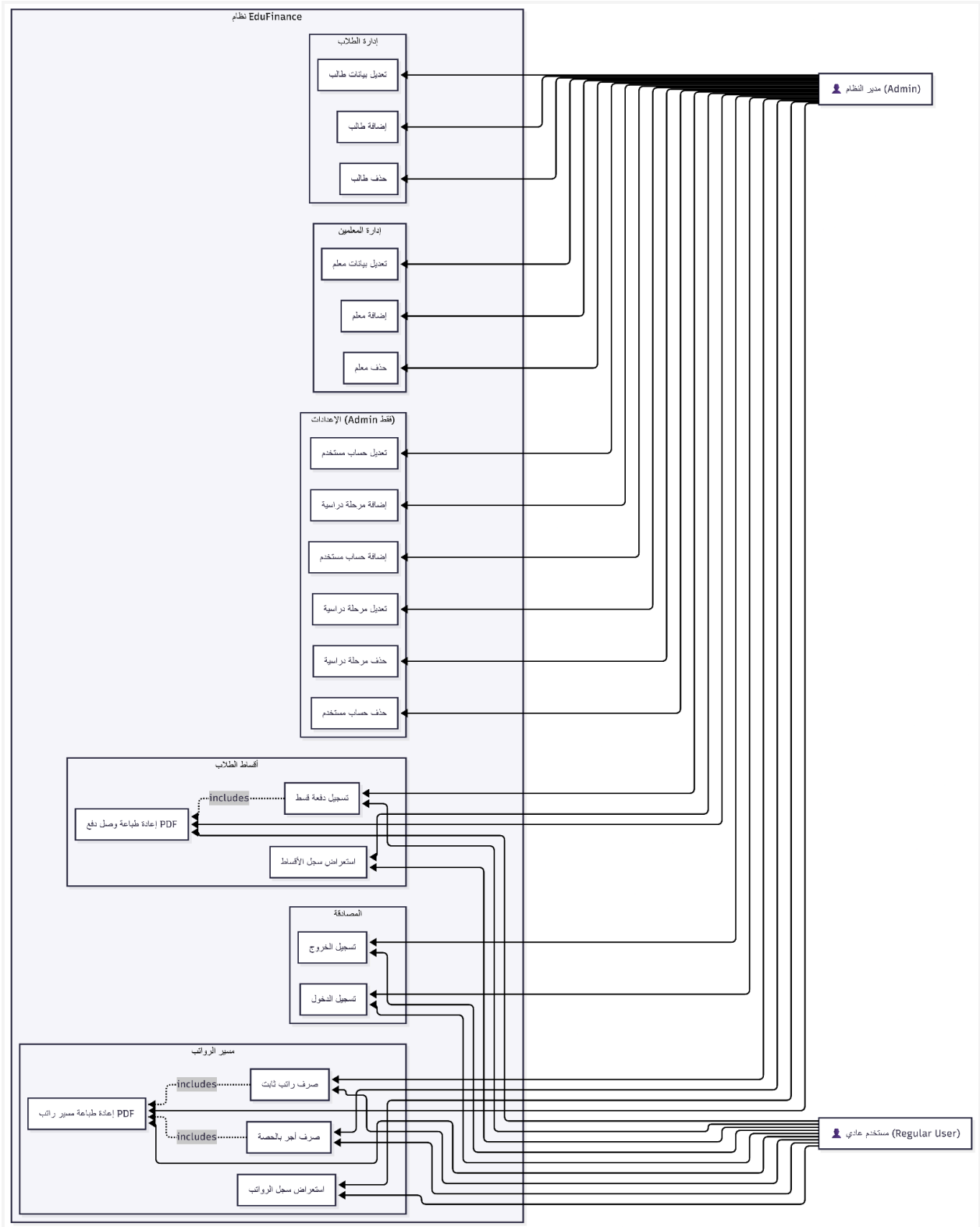
لضمان اتساق البيانات وسلامتها من التلف (Data Integrity)، يعتمد النظام على تفعيل مفهوم المعاملات الذرية (Atomic Transactions) في قاعدة البيانات عند إجراء السداد المالي للطلاب:

1. يقوم المستخدم بإدخال مبلغ القسط المالي والضغط على زر السداد في الواجهة.
2. يستقبل StudentPaymentsController الحدث ويقوم بالتحقق من صحة المدخلات ومقارنتها بالمبلغ المتبقي على الطالب.
3. يتم إيقاف التثبيت التلقائي عبر قاعدة البيانات conn.setAutoCommit(false) لبدء المعاملة الموحدة.
4. يتم تنفيذ الاستعلام الأول: تحديث جدول الطلاب وإضافة القسط المدفوع UPDATE students SET ....paid\_amount
5. يتم تنفيذ الاستعلام الثاني: إدراج عملية مالية جديدة في جدول السجلات ....INSERT INTO transactions
6. في حال نجاح العمليتين معاً يتم تأكيد وحفظ المعاملة نهائياً (conn.commit())، وفي حال حدوث أي استثناء أو فشل يتم التراجع الفوري عن كافة التغييرات لحماية الحسابات (conn.rollback()).
7. يستدعي النظام فئة PdfReceiptGenerator لإنشاء وصل دفع PDF احترافي وتحديث جداول العرض المرئية فوراً.

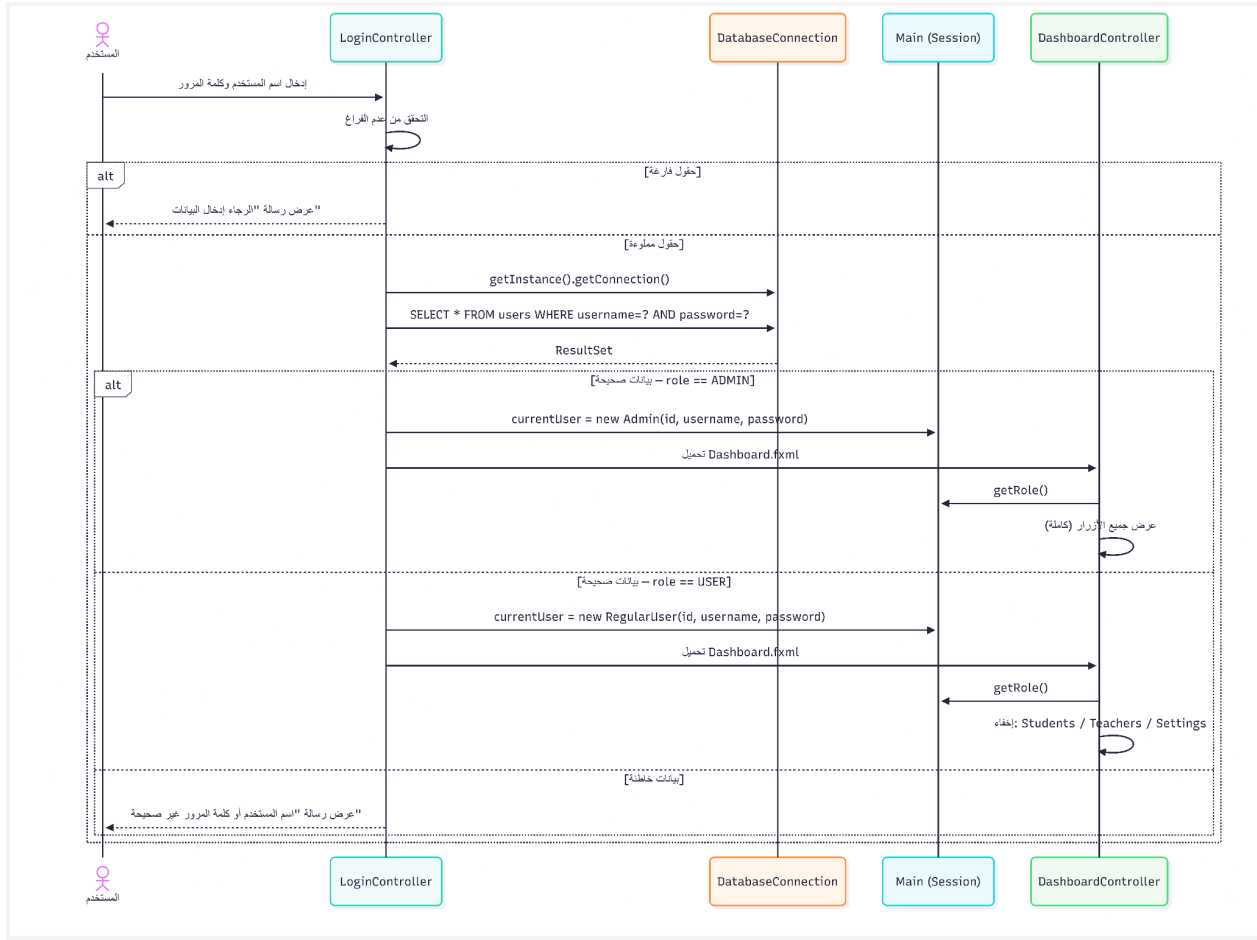
## 6. التوثيق المخططي الكامل للنظام (Comprehensive UML) (Diagrams Reference)



مخطط الفئات الشامل (Class Diagram)

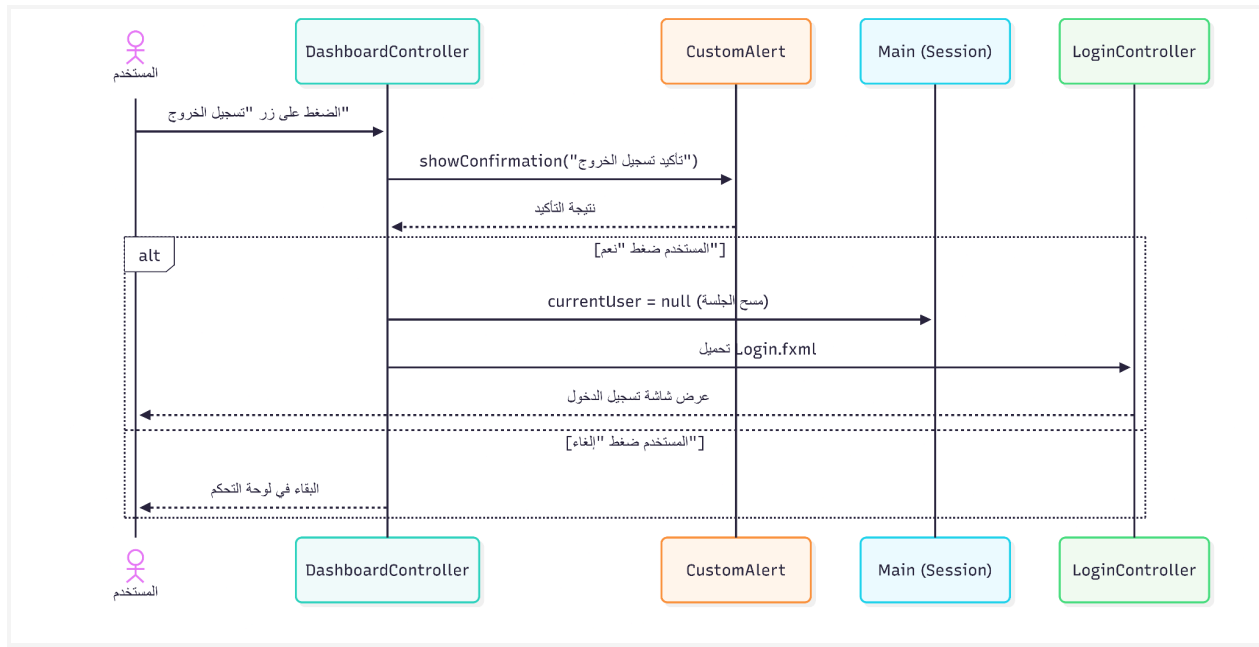


مخطط حالات الاستخدام (Use Case Diagram)

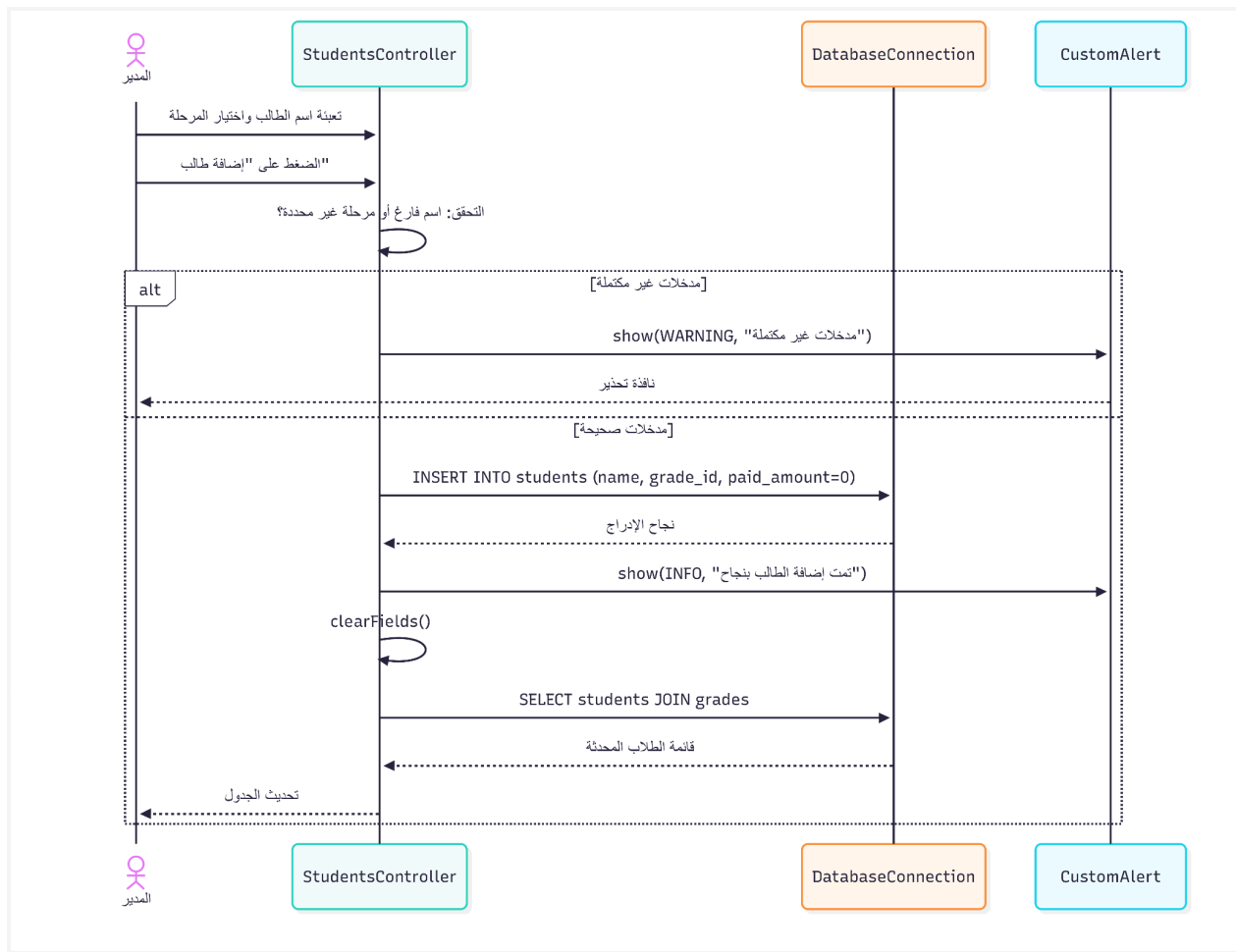


مخطط تتابع تسجيل الدخول (Sequence Diagram - Login)

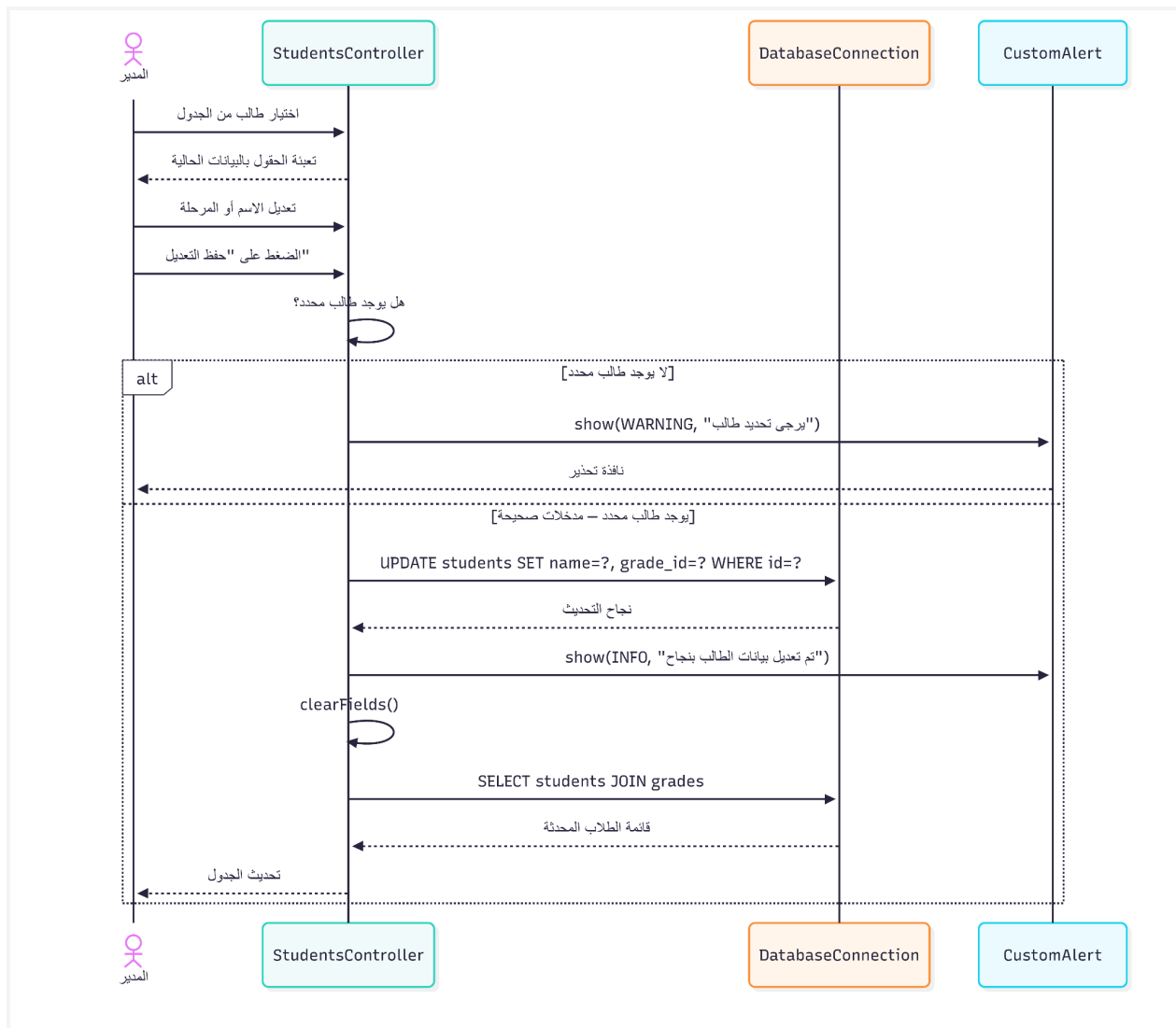




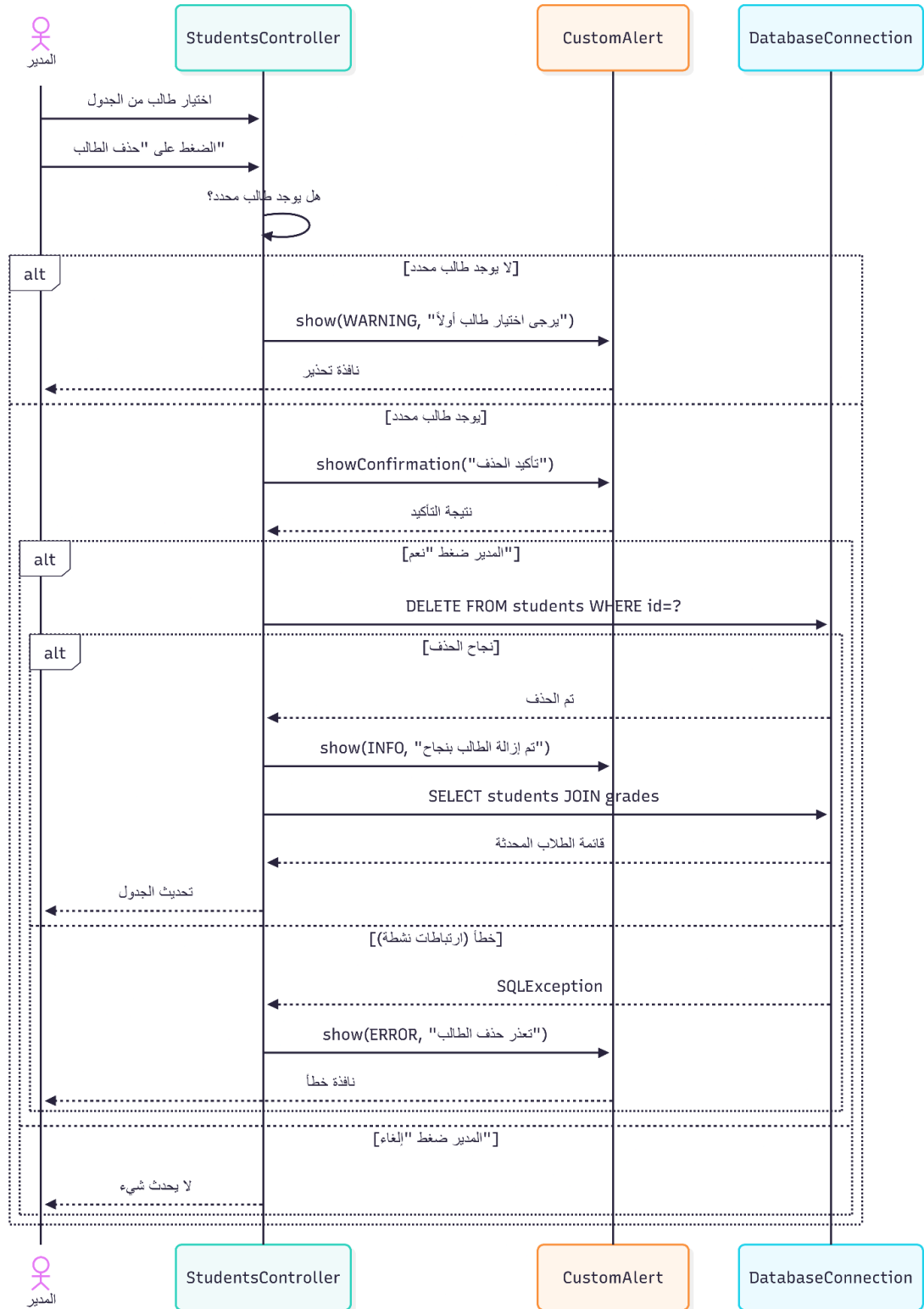
مخطط تتابع تسجيل الخروج (Sequence Diagram - Logout)



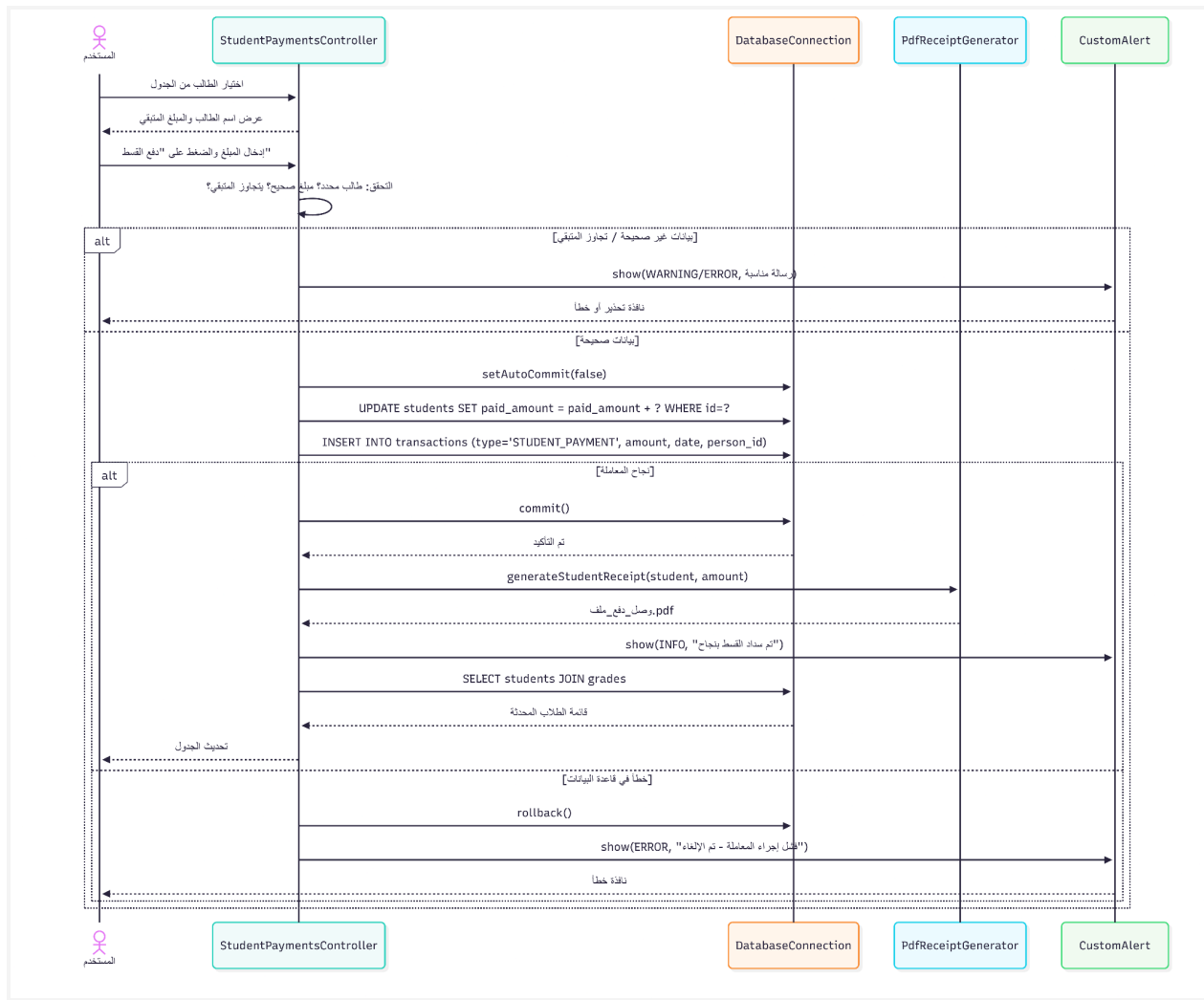
مخطط تتابع إضافة طالب (Sequence Diagram - Add Student)



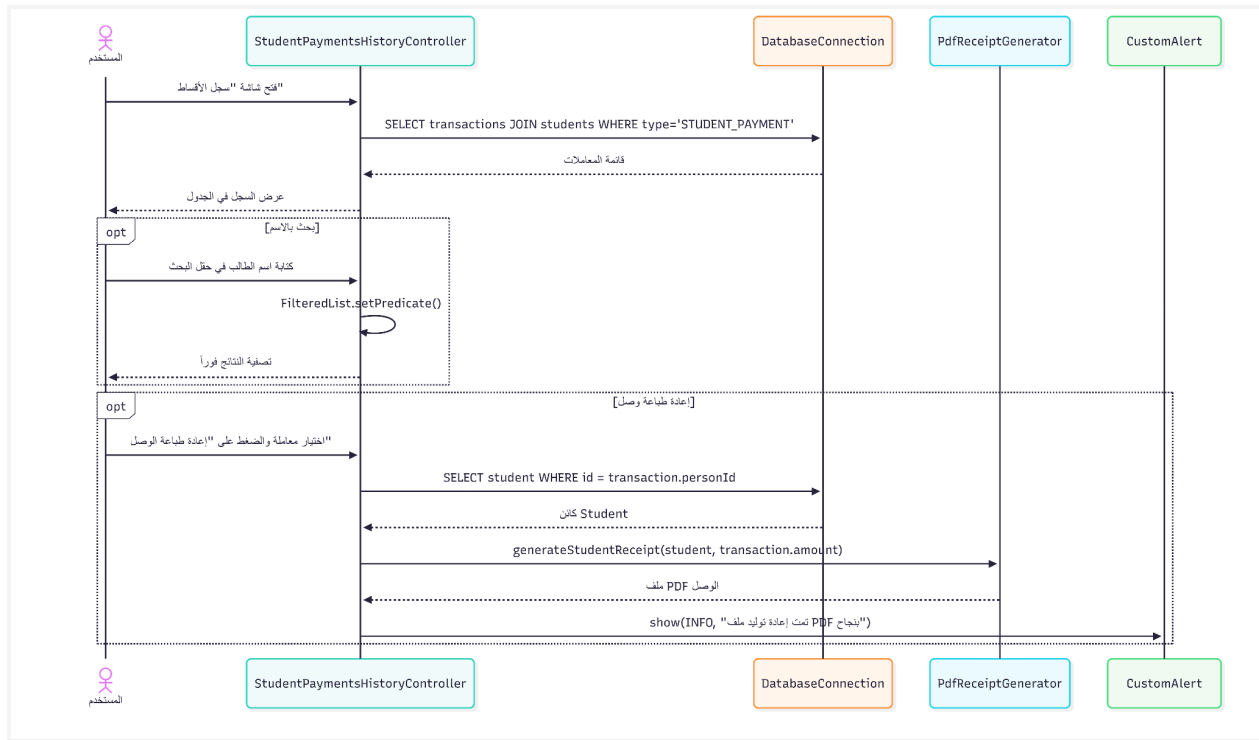
مخطط تتابع تعديل بيانات طالب (Sequence Diagram - Edit Student)



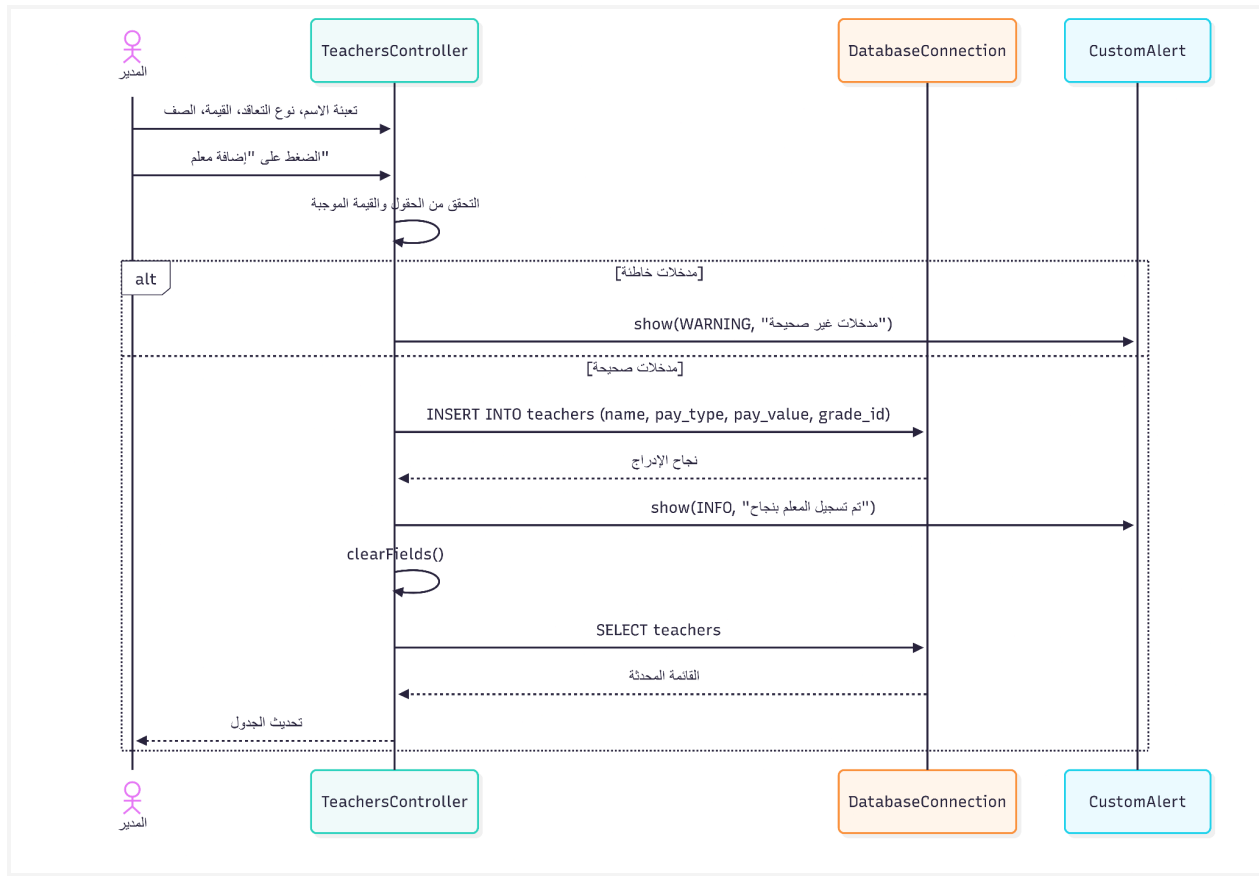
مخطط تتابع حذف طالب (Sequence Diagram - Delete Student)



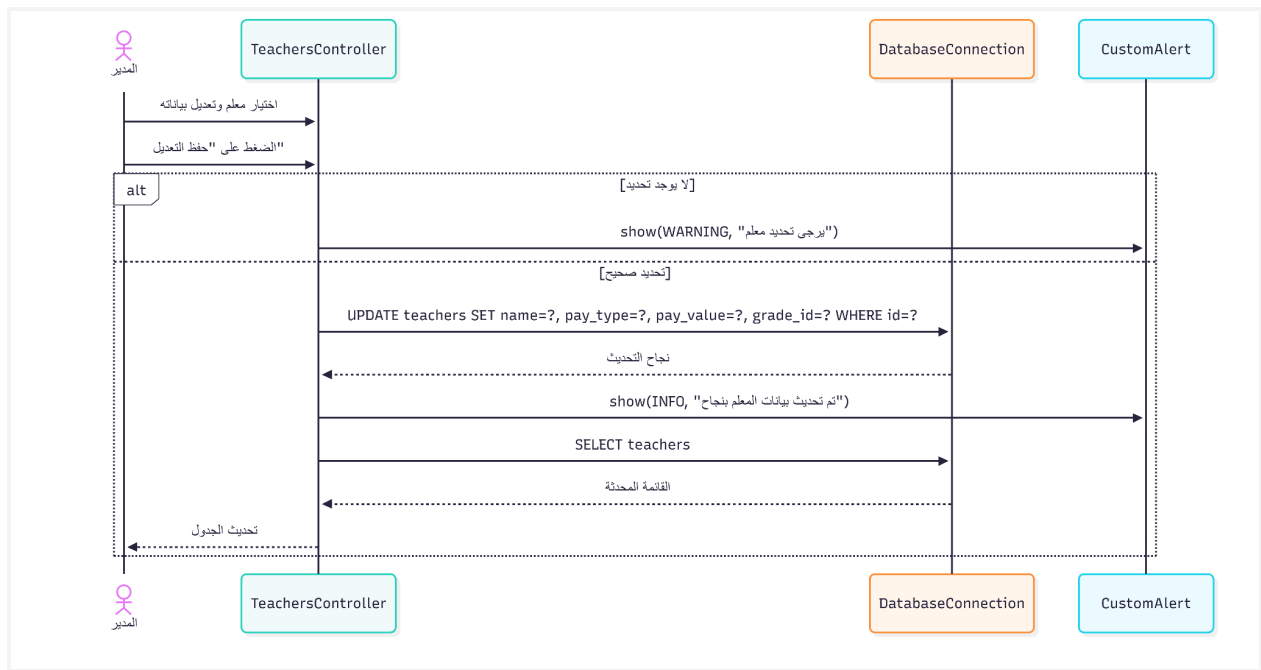
مخطط تتابع تسجيل قسط طالب (Sequence Diagram - Pay Installment)



مخطط استعراض سجل أقساط الطلاب وإعادة الطباعة

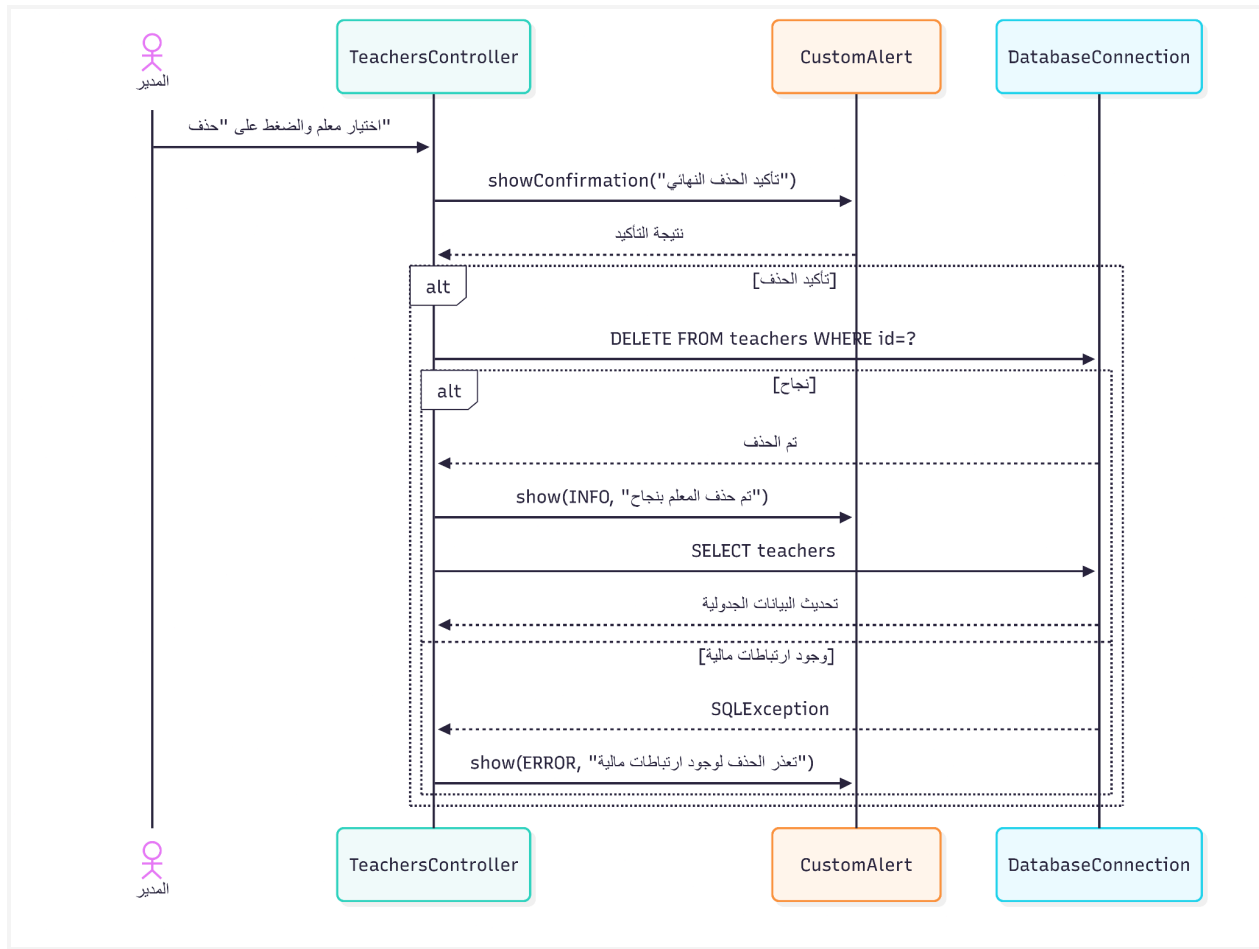


مخطط تتابع إضافة معلم (Sequence Diagram - Add Teacher)

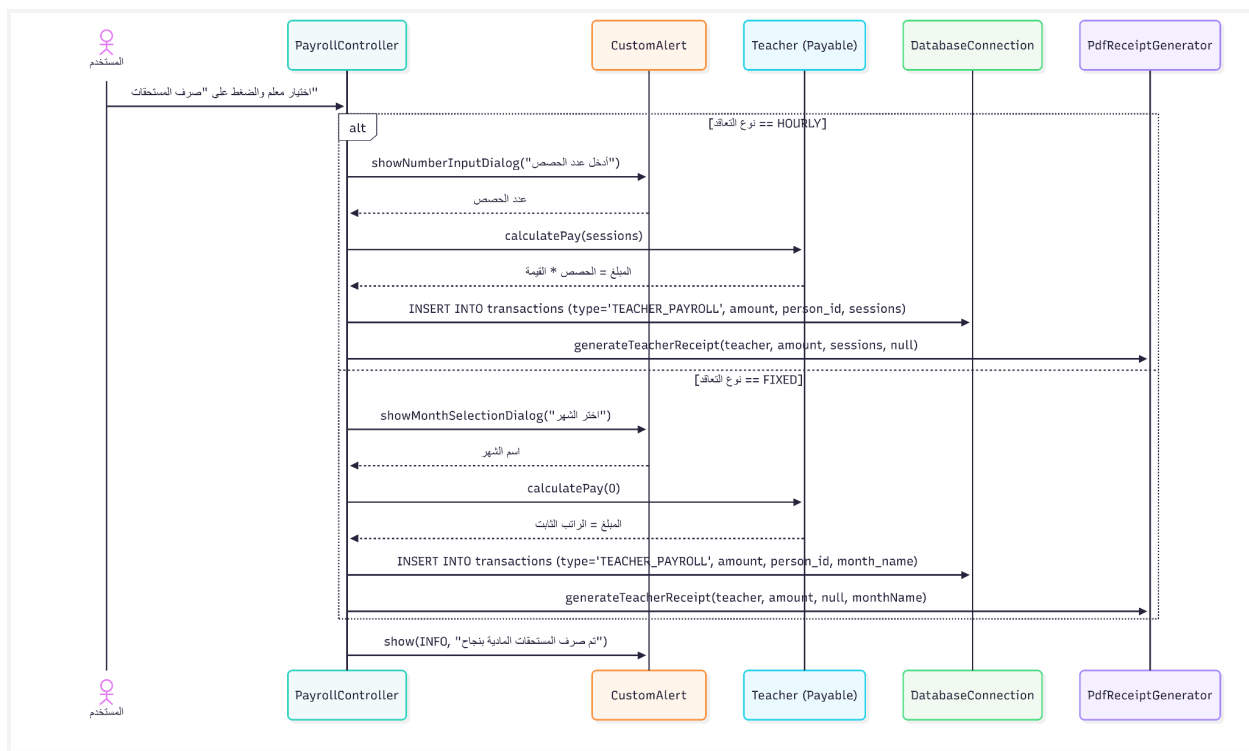


مخطط تتابع تعديل بيانات معلم (Sequence Diagram - Edit Teacher)

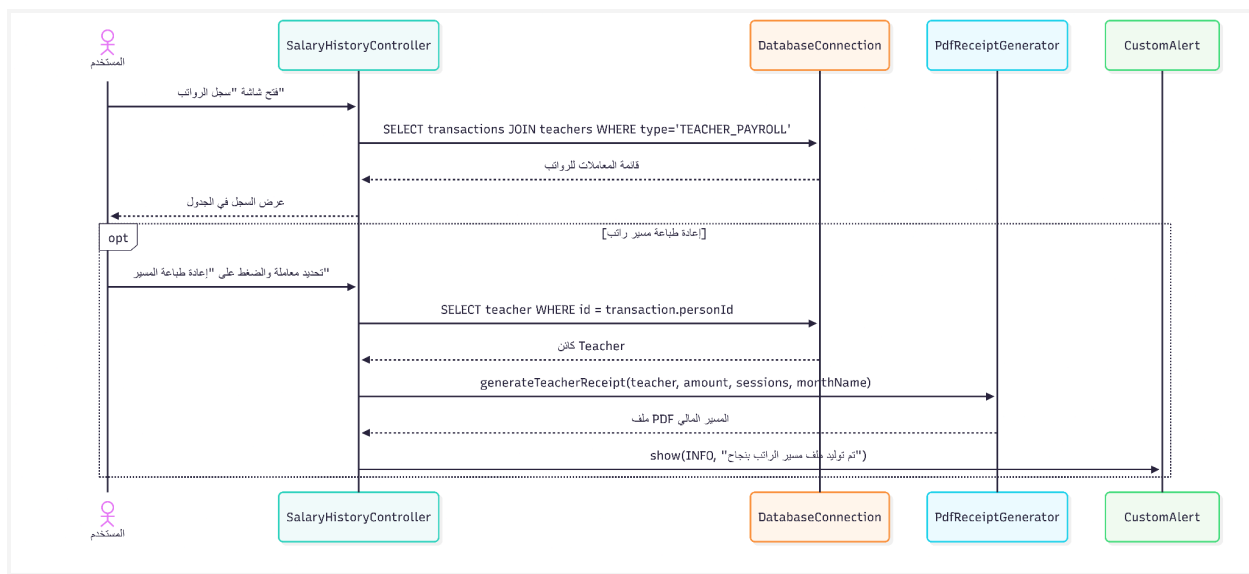




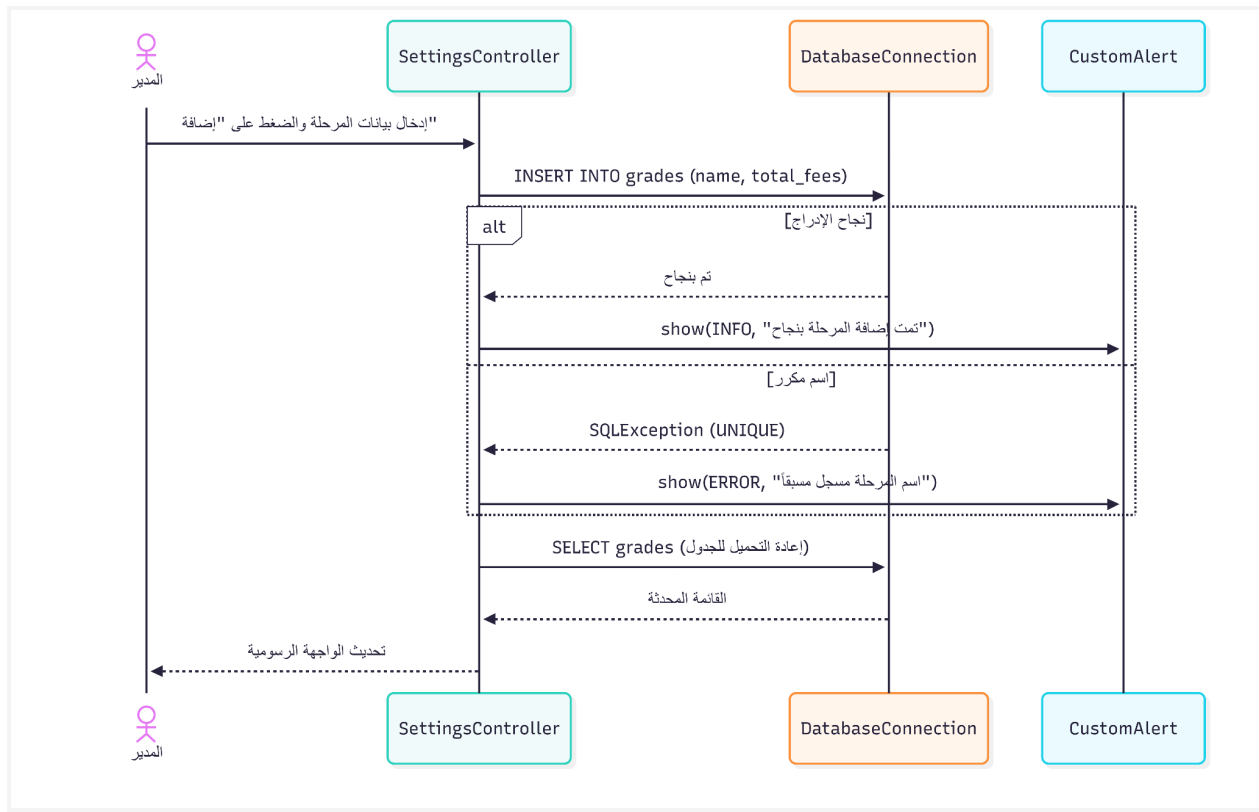
مخطط تتابع حذف معلم (Sequence Diagram - Delete Teacher)



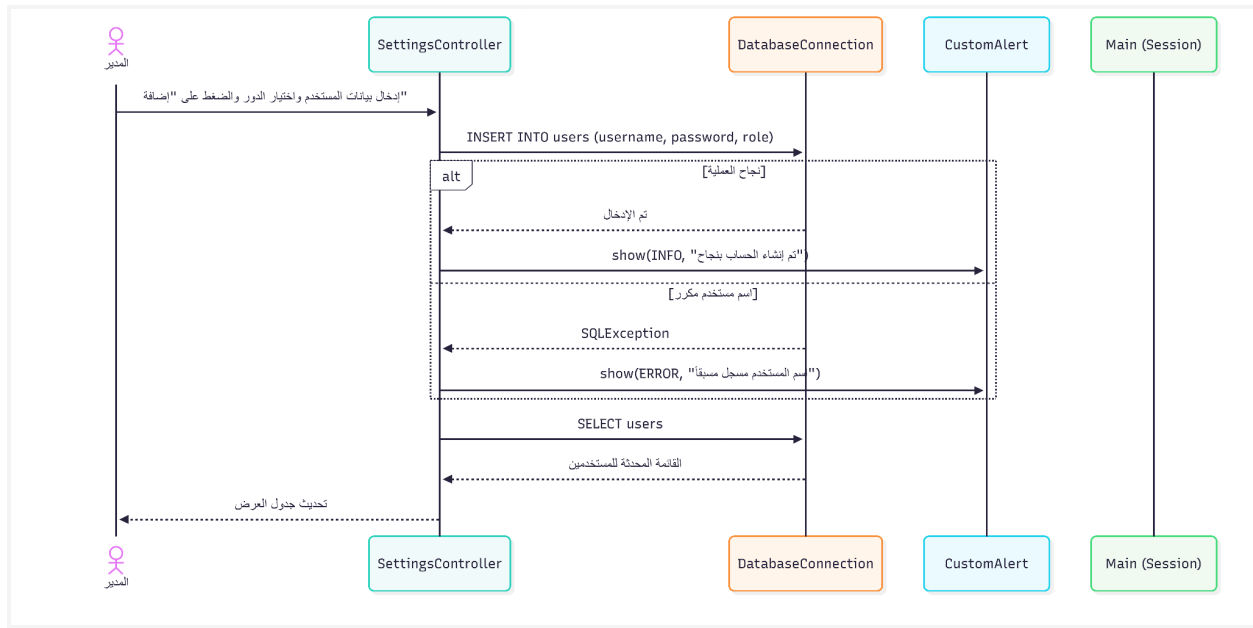
مخطط تتابع صرف راتب معلم (Sequence Diagram - Process Payroll)



مخطط استعراض سجل الرواتب وإعادة الطباعة



مخطط إدارة المراحل الدراسية (Settings — Grades)



مخطط إدارة حسابات المستخدمين (Settings — Users)

## 7. أمان البيانات والتحكم بالوصول (Security & Access Control)

- لحماية المعطيات المالية الحساسة للمؤسسة التعليمية، يعتمد نظام EduFinance استراتيجيتين أساسيتين للحماية:
- **منع هجمات حقن قواعد البيانات (SQL Injection Prevention):** يتم الاستعلاء من قاعدة البيانات المحلية SQLite بشكل مطلق باستخدام كائنات PreparedStatement المجهزة مسبقاً والممررة عبر معالم (Parameters)، مما يضمن معاملة المدخلات كنصوص مجردة وليس كأوامر برمجية قابلة للتنفيذ.
- **سلامة القيود المرجعية (Referential Integrity):** يتم تشغيل آلية المفاتيح الأجنبية بشكل إلزامي عبر أمر قاعدة البيانات PRAGMA foreign\_keys = ON؛ لضمان عدم حذف مرحلة دراسية أو مستخدم مرتبط بعمليات مالية حية، مما يحمي قاعدة البيانات من الوصول إلى حالات غير متسقة.

## 8. الخلاصة الهندسية

يظهر نظام EduFinance توافقاً هندسياً متميزاً مع أفضل ممارسات وتصميم هندسة البرمجيات الاحترافية. من خلال دمج نمط المعمارية MVC مع أنماط التصميم المتقدمة وتطبيقات OOP الصارمة، يحقق النظام توازناً مثالياً بين الأداء العالي والأمان وسهولة الصيانة الممتدة، مما يجعله نموذجاً برمجياً متكاملًا جديرًا بالتوثيق والدراسة الأكاديمية.